

```
def await[F[_],A,O] (
  req: F[A]) (
  rcv: Either[Throwable,A] => Process[F,O]): Process[F,O] =
  Await(req, rcv)
```

Again, using ++, we define flatMap. This is another combinator where we must take care to ensure exception safety—we don’t know for sure whether *f* will throw an exception, so we wrap calls to *f* in Try once again. Other than that, the definition looks very similar to what we wrote before:

```
def flatMap[O2](f: O => Process[F,O2]): Process[F,O2] =
  this match {
    case Halt(err) => Halt(err)
    case Emit(o, t) => Try(f(o)) ++ t.flatMap(f)
    case Await(req,rcv) =>
      Await(req, rcv andThen (_ flatMap f))
  }
```

Let’s see what else we can express with this new Process type. The *F* parameter gives us a lot of flexibility.

15.3.1 Sources

Before, we had to write a separate function to drive a process forward while reading from a file. Now, we can represent an effectful source directly using a `Process[IO,O]`.⁷

To see how `Process[IO,O]` is indeed a source of *O* values, consider what the `Await` constructor looks like when we substitute `IO` for *F*:

```
case class Await[A,O] (
  req: IO[A],
  rcv: Either[Throwable, A] => Process[IO,O]
) extends Process[IO,O]
```

Thus, any requests of the “external” world can be satisfied just by running or flatMapping into the `IO` action *req*. If this action returns an *A* successfully, we invoke the *rcv* function with this result, or a `Throwable` if *req* throws one. Either way, the *rcv* function can fall back to another process or clean up any resources as appropriate. Here’s a simple interpreter of an I/O Process that collects all the values emitted.

Listing 15.6 The runLog function

```
def runLog[O](src: Process[IO,O]): IO[IndexedSeq[O]] = IO {
  val E = java.util.concurrent.Executors.newFixedThreadPool(4)
  @annotation.tailrec
  def go(cur: Process[IO,O], acc: IndexedSeq[O]): IndexedSeq[O] =
    cur match {
      case Emit(h,t) => go(t, acc :+ h)
      case Halt(End) => acc
      case Halt(err) => throw err
      case Await(req,rcv) =>
```

⁷ There are some issues with making this representation resource-safe that we’ll discuss shortly.

```

    val next =
      try recv(Right(unsafePerformIO(req)(E)))
      catch { case err: Throwable => recv(Left(err)) }
    go(next, acc)
  }
  try go(src, IndexedSeq())
  finally E.shutdown
}

```

An example usage of this is to enumerate all the lines in a file:

```

import java.io.{BufferedReader, FileReader}
val p: Process[IO, String] =
  await(IO(new BufferedReader(new FileReader("lines.txt")))) {
    case Right(b) =>
      lazy val next: Process[IO, String] = await(IO(b.readLine)) {
        case Left(e) => await(IO(b.close))(_ => Halt(e))
        case Right(line) =>
          if (line eq null) Halt(End)
          else Emit(line, next)
      }
      next
    case Left(e) => Halt(e)
  }

```

Handle forcible
termination or
termination due
to an error.

The `readLine` function
on `BufferedReader`
returns `null` when it
reaches the end of file.

Then we can use `runLog(p)` to get all the lines in the file `lines.txt` as an `IO[IndexedSeq[String]]`.

Note that we're making sure that the file is closed regardless of how the process terminates. In section 15.3.2, we'll discuss how to ensure that all such processes close the resources they use (they're *resource-safe*), and discover a few generic combinators for ensuring resource safety.



EXERCISE 15.10

The `runLog` function can be defined more generally for any `Monad` in which it's possible to catch and raise exceptions (for instance, the `Task` type mentioned in chapter 13, which adds this capability to the `IO` type). Define this more general version of `runLog`. Note that this interpreter can't be tail-recursive, and relies on the underlying monad for stack safety.

```

trait Process[F[_], O] {
  def runLog(implicit F: MonadCatch[F]): F[IndexedSeq[O]]
  ...
}

trait MonadCatch[F[_]] extends Monad[F] {
  def attempt[A](a: F[A]): F[Either[Throwable, A]]
  def fail[A](t: Throwable): F[A]
}

```